Designing an Interactive Educational Framework for LZSCC.111

Andrei Florea (39603784) • Supervisor: Tobias Meggendorfer • 29th October 2026

Abstract

Students on LZSCC.111 Software Development learn to program in C, where program state is hidden and mistakes often show up only as a wrong number or a crash. This project will build a framework of interactive C exercises that increase in difficulty. In each exercise the student implements a single function. Provided scaffolding handles input and output, and a Python visualiser shows what the code does step by step, such as the bars of an array being searched or a robot moving through a maze. Student code runs inside a C harness that records each step as an event trace and catches crashes, infinite loops and out-of-bounds access. The framework will be evaluated technically and through a small study with students, comparing the exercises with and without the visualiser. The expected outcome is a reusable set of tasks for LZSCC.111, along with indicative evidence on whether visual feedback improves novice understanding and engagement.

1 Introduction

LZSCC.111 introduces students to programming in C. For a beginner, C gives little visible feedback [1]. A loop that searches an array or a function that moves a robot produces at most a line of printed output, and the student cannot see how the program reached its result [2]. Reading input, parsing it and formatting output also take effort unrelated to the concept being taught, so beginners spend their time on scanf and printf instead of the algorithm. Research on algorithm visualisation has found that showing execution helps most when students interact with the visualisation rather than only watch it [3], [4]. This project applies that idea to LZSCC.111. Students write only the core logic of each task in C, and a Python visualiser shows its effect. The rest of this proposal is organised as follows. Section 2 reviews related work and the gap this project addresses. Section 3 sets out the aims, objectives and research questions, followed by the methodology. Section 4 describes the programme of work, Section 5 relates to the resources required, Section 6 reveals the risks and contingencies, and last but not least, Section 7 lists the references.

2 Background and Related Work

Show where your project fits into existing work: similar projects, tools, systems, and papers **example-article**, **example-inproceedings**. *This is an integrated synthesis, not a list of who did what. For each source, explain why it is relevant to your project, what its limitations are ('however, . . . '), and how your project addresses them ('therefore, . . . ').*

Organise the section thematically, chronologically, methodologically, or as a funnel (general background → closely related work). End with the scope of your project: what exactly you will do, and what not.

3 Proposed Project

3.1 Aims and Objectives

The aim of the project is to build a reusable framework of visual C tasks of increasing difficulty, aligned with the LZSCC.111 curriculum, and to gather evidence on whether it helps students learn. The objectives are:

O1 Objective one – Literature review on program visualisation, scaffolding and novice programming [5].

O2 Objective two – Design the architecture that connects students' C code to Python visualiser through an event-trace format.

O3 Objective three – Build a C harness that runs code safely, catching crashes, infinite loops and out-of-bounds errors then reporting them in plain language.

O4 Objective four – Implementing at least 4 tasks with different difficulty, each with C scaffolding, a solution and a visualisation.

O5 Objective five – Automated tests for the framework so that new tasks can be added.

O6 Objective six – Evaluate the framework technically and through a small user study.

3.2 Methodology

The project will follow an iterative approach, with a working increment shown to the supervisor every week. This suits the project because each task can be built, tried and adjusted on its own, and early feedback on the first task will shape the design of the rest. A waterfall approach was considered and rejected, since the requirements for later tasks depend on what is learned from the first prototype.

The framework separates student code from visualisation through an event trace. Students solve each task by calling scaffolding functions, such as one that compares two elements or moves the robot, and each function records its action as an event (e.g., "compared index 3"). The C harness writes these events as JSON lines to a separate file, so the student's printf output does not interfere. The Python visualiser then replays the trace as an animation. Since events are flushed as they occur, a crashing program still leaves a trace up to the point of failure (O3). The trace also lets students step backwards [6] and makes visualisations testable: automated tests compare it against the trace of a reference solution [7](O5).

3.3 Evaluation

Your plan for analysis: how will you evaluate the results? Name concrete metrics, baselines, benchmarks, test suites, or study designs, and say how they relate to the objectives in Section 3.1.

3.4 Ethics

The user study will aim for 10 to 15 participants, current LZSCC.111 students. This size is realistic for the timeframe and enough for a descriptive comparison, though not for statistically results. Participants will receive an information sheet and sign a consent form, and all responses will be anonymised. Ethics approval will be requested early in Michaelmas term so that the study can be done in the Lent term.

4 Programme of Work

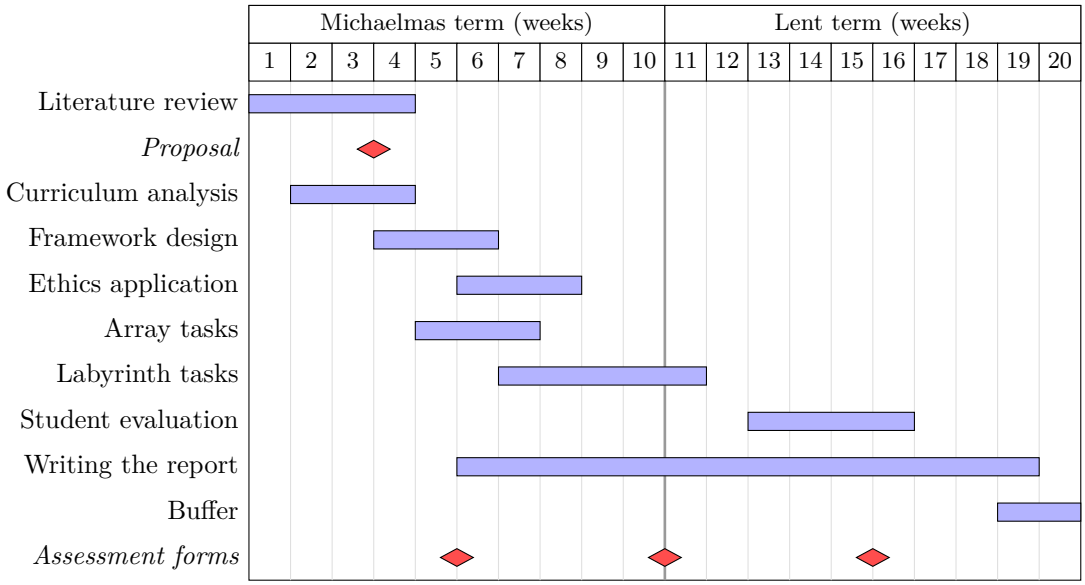


Figure 1: Project schedule. There is a four-week break between weeks 10 and 11.

5 Resources Required

- **Software:** gcc/clang, Python, pygame and Git are all free and open-source, and I will install them myself. A free personal GitHub account (username: florea2005). Overleaf: a free account, or access through Lancaster's email.
- **Hardware:** My personal laptop will be used for development, testing and writing the report. No special hardware or GPU is needed. Lancaster University Leipzig's lab machines will be used during testing to confirm that the tasks run in the real LZSCC.111 environment.

6 Risks and Contingency

Table 1: Risks and mitigation.

Risk	Likelihood	Impact	Mitigation
Too few participants for the study	low	high	run it during a workshop
C-Python communication is harder than expected	medium	high	build a minimal prototype with only array task first
Illness or other absence	low	medium	Buffer time in the schedule

References

[1] P. A. Smith and G. I. Webb, 'The efficacy of a low-level program visualisation tool for teaching programming concepts to novice C programmers,' *Journal of Educational Computing Research*, vol. 22, no. 2, 2000. DOI: 10.2190/N0VV-0P48-XJ9G-F8WV

[2] S. Fincher et al., 'Notional machines in computing education: The education of attention,' in *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '20)*, ACM, 2020, pp. 21–50. DOI: 10.1145/3437800.3439202

[3] C. D. Hundhausen, S. A. Douglas and J. T. Stasko, 'A meta-study of algorithm visualisation effectiveness,' *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002. [Online]. Available: https://www.academia.edu/15636659/A_Meta_Study_of_Algorithm_Visualization_Effectiveness

[4] T. L. Naps et al., 'Exploring the role of visualisation and engagement in computer science education,' *ACM SIGCSE Bulletin*, vol. 35, no. 2, pp. 131–152, 2003. DOI: 10.1145/782941.782998

[5] J. Sorva, V. Karavirta and L. Malmi, 'A review of generic program visualisation systems for introductory programming education,' *ACM Transactions on Computing Education*, vol. 13, no. 4, 2013. DOI: 10.1145/2490822

[6] P. J. Guo, 'Online Python tutor: Embeddable web-based program visualization for CS education,' in *Proceedings of the 44th ACM Technical Symposium on Computer Science Education (SIGCSE '13)*, ACM, 2013, pp. 579–584. DOI: 10.1145/2445196.2445368

[7] P. Ihantola, T. Ahoniemi, V. Karavirta and O. Seppälä, 'Review of recent systems for automatic assessment of programming assignments,' in *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, ACM, 2010, pp. 86–93. DOI: 10.1145/1930464.1930480